

GARUDA SoC

BLOCK-LEVEL DESIGN SPECIFICATION

Block II — Domain-Specific Unit

Three 16×16 MAC, 48-bit Accumulator, Custom-0 ISA Extension

Document ID: AERO-GARUDA-DS-002

Block ID: U_DSU

Revision: 1.0

Status: Released for RTL

Process Node: 28 nm

Target Frequency: 200 MHz (100 MHz fallback)

Parent Document: GARUDA SoC TRM v2.0

Author: Team AeroSoC

Contents

I	Introduction	1
I.I	Purpose	1
I.II	Scope	1
I.III	Conventions	1
I.III.1	Signal naming	1
I.III.2	Bit ordering	1
I.III.3	Numerical formats	2
I.III.4	Hierarchy	2
I.III.5	Gate counts	2
I.IV	Reference Documents	2
II	DSU Top Module	3
II.I	Block as Black Box	3
II.II	Input Pins	3
II.II.1	Infrastructure signals	3
II.II.2	Operand buses	4
II.II.3	Control bundle	4
II.III	Output Pins	4
II.III.1	Pipeline control	5
II.III.2	Writeback path	5
II.III.3	Debug observation	5
II.III.4	Illegal-instruction trap	5
II.IV	Module Pin Summary	5
III	DSU Internal Architecture	6
III.I	Block Inventory	6
III.I.1	The nine blocks	6
III.I.2	Glue logic	7
III.I.3	What does not exist inside the DSU	7
III.II	Signal Flow	7
III.II.1	Input-side paths	8
III.II.2	Output-side paths	8
III.III	The Aux Path	9
III.IV	Stall FSM Behaviour	9
IV	MAC Cluster	10
IV.I	Purpose	10
IV.II	Cluster Architecture	10
IV.II.1	Block inventory	10
IV.III	Cluster Pin Contract	11
IV.IV	Per-MAC Enable Decoder	11

IV.V Saturation Writeback Routing	11
IV.V Signal Flow Inside the Cluster	11
IV.VI.1 Broadcast signals	11
IV.VI.2 Per-MAC signals	12
IV.VI.3 Outputs	12
IV.VI Cluster Gate Count	12
V Single MAC Unit	14
V.I Function	14
V.II Pin Contract	14
V.III Internal Architecture	15
V.III.1 Block inventory	15
V.IV Absolute-Value Pre-Stages	16
V.V Operand Router	16
V.VI Two Multipliers and the 3:2 Compressor	16
V.VII Product Register	16
V.VIII Accumulator-Write Input Mux	16
V.IX Accumulator Register	17
V.X Cycle-by-Cycle Behaviour	17
V.XI Flush Handling	17
V.XII Power Management	17
VI The 16×16 Signed Multiplier	19
VI.I Architecture Choice	19
VI.II Pin Contract	19
VI.III Stage 1 — Booth Radix-4 Encoder	19
VI.IV Stage 2 — Partial Product Generation	19
VI.V Stage 3 — Dadda Compression Tree	20
VI.VI Stage 4 — Final 32-bit Kogge-Stone Adder	20
VI.VII Timing	21
VI.VIII Multiplier Instances in the DSU	21
VII The 48-bit Accumulator Adder	22
VII.I Function	22
VII.II Pin Contract	22
VII.II.1 Two-input mode	22
VII.II.2 Three-input mode	23
VII.III Parallel-Prefix Structure	23
VII.III.1 Phase 1 — Pre-processing	23
VII.III.2 Phase 2 — Prefix network	23
VII.III.3 Phase 3 — Post-processing	24
VII.IV CSA Pre-Stage for MACDOT	24
VII.V Total Gate Count	24
VII.VI Timing	24
VIII DSU Auxiliary Blocks	25
VIII.I Barrel Shifter	25
VIII.I.1 Pin contract	25
VIII.I.2 Architecture	25
VIII.I.3 Gate count	25
VIII.I.4 Timing	26
VIII.I.5 Output width	26

VIII.I	Saturation Unit	26
VIII.II	Pin contract	26
VIII.II.A	Architecture	26
VIII.II.B	Gate count	26
VIII.III	Read-back Multiplexer	26
VIII.III	Pin contract	26
VIII.III.A	Architecture	26
VIII.III.B	Gate count	27
VIII.IV	Result Selector Mux	27
VIII.IV	Pin contract	27
VIII.IV.A	Architecture	27
VIII.IV.B	Gate count	27
VIII.V	Stall FSM	28
VIII.V	Pin contract	28
VIII.V.A	State machine	28
VIII.V.B	Gate count	28
VIII.VI	Overflow Sticky Flag	28
VIII.VI	Pin contract	28
VIII.VII	Logic	29
VIII.VII	Gate count	29
VIII.VIII	Summary	29
IX	Custom-0 Instruction Set	30
IX.I	Encoding Format	30
IX.I.1	Bit field layout	30
IX.I.2	Field semantics	30
IX.II	funct5 Assignments	30
IX.III	Accumulator Select	30
IX.IV	Instruction Reference	30
IX.IV.1	MAC_SEL — Multiply-Accumulate	30
IX.IV.2	MACSUB — Multiply-Subtract	30
IX.IV.3	MACCLEAR — Clear Accumulator	30
IX.IV.4	MACLOAD — Load Accumulator	31
IX.IV.5	MACREAD_LO — Read Accumulator Low Half	31
IX.IV.6	MACREAD_HI — Read Accumulator High Half	31
IX.IV.7	MACDOT — Dot-Product Multiply-Accumulate	31
IX.IV.8	MACSHIFT — Arithmetic Right-Shift Accumulator	31
IX.IV.9	MACSAT — Saturate Accumulator	31
IX.IV.10	MACABS — Absolute-Value Multiply-Accumulate	31
IX.V	Illegal-Instruction Conditions	31
IX.VI	Programming Example: APF Inner Loop	31
X	Verification, Constraints, and Sign-Off	35
X.I	Timing Constraints	35
X.I.1	Critical paths	35
X.I.2	Fallback frequency	35
X.II	Power Management	35
X.II.1	Clock gating policy	35
X.II.2	Estimated power	35
X.III	Verification Hooks	36
X.III.1	Built-in observability	36
X.III.2	Required directed tests	36

X.III.3 Reference model	36
X.IV Gate Count Summary	36
X.V Sign-Off Checklist	37

List of Figures

II.1 DSU top module — external pin view.	3
III.1 DSU internal block diagram. Solid arrows are data buses, dashed arrows are control signals.	6
IV.1 MAC cluster internal architecture. Lighter arrows are the always-live debug taps; they bypass the output mux.	11
V.1 Single MAC internal datapath under MACDOT Option A. Two multipliers feed a 3:2 compressor that produces a carry-save pair; the product register holds both during cycle 1; the 48-bit adder produces the new accumulator value in cycle 2. .	15

List of Tables

II.1 DSU top-module input signals.	4
II.2 DSU top-module output signals.	5
III.1 Decoder output signals and their destinations.	7
IV.1 MAC cluster external pin contract.	12
IV.2 Per-MAC enable decoder truth table.	13
IV.3 MAC cluster gate count breakdown (pre-synthesis estimates).	13
V.1 Single MAC unit pin contract.	14
V.2 Operand router truth table.	16
V.3 Accumulator-input mux selection.	17
V.4 Per-instruction cycle-by-cycle behaviour inside one MAC.	18
VI.1 16×16 signed multiplier pin contract.	19
VI.2 Booth Radix-4 encoding table.	20
VI.3 Dadda compression tree levels for nine partial products.	20
VI.4 16×16 multiplier gate count breakdown.	21
VII.148-bit Kogge-Stone accumulator-adder pin contract.	22
VII.2Kogge-Stone prefix network for 48-bit addition.	23
VII.348-bit accumulator-adder gate count.	24
VIII.1Barrel shifter pin contract.	25
VIII.2Saturation unit pin contract.	26
VIII.3Read-back mux pin contract.	27
VIII.4Result selector mux pin contract.	27
VIII.5Stall FSM pin contract.	28
VIII.6Overflow sticky flag pin contract.	28
VIII.7Auxiliary block gate-count summary.	29
IX.1 funct5 instruction encoding.	31
IX.2 acc field encoding.	31
IX.3 MAC_SEL instruction reference.	32
IX.4 MACSUB instruction reference.	32
IX.5 MACCLEAR instruction reference.	33
IX.6 MACLOAD instruction reference.	33
IX.7 MACREAD_LO instruction reference.	33
IX.8 MACREAD_HI instruction reference.	33
IX.9 MACDOT instruction reference.	34
IX.10MACSHIFT instruction reference.	34
IX.11MACSAT instruction reference.	34

IX.12MACABS instruction reference.	34
X.1 Required directed tests.	36
X.2 Total DSU gate-count rollup (pre-synthesis estimates).	37

BLOCK I

Introduction

I.I Purpose

This document specifies the Domain-Specific Unit (DSU), Block II of the GARUDA SoC. The DSU is the primary acceleration engine for the Artificial Potential Field (APF) collision-avoidance workload and the matrix-multiply path of the 9-state Extended Kalman Filter (EKF). It is implemented as an ISA extension embedded in the CPU execute stage rather than a bus-attached accelerator.

This is a block-level design specification. It defines the external pin contract, internal architecture, signal flow, instruction encoding, and verification hooks of the DSU. The RTL team builds against this document; the firmware team writes against the instruction set in Chapter IX.

I.II Scope

This document covers:

- The DSU top-module interface to the RV32IM core.
- All internal sub-blocks of the DSU, walked top-down to gate-level granularity.
- The 10-instruction Custom-0 ISA extension implemented by the DSU.
- Pipeline integration including stall, flush, and trap behaviour.
- Reset, clocking, and power-management policy.

This document does *not* cover:

- The RV32IM core itself (Block I) beyond the DSU's interface.
- Memory subsystem, bus fabric, peripherals (Blocks III through XXIV).
- Physical design, place-and-route, or backend flow.
- Software (firmware) implementation of APF, EKF, or PID.

I.III Conventions

I.III.1 Signal naming

Signal names use lowercase with underscores. Buses are denoted with their full width in brackets, e.g. `rs1_data[31:0]`. Active-low signals carry an `_n` suffix.

I.III.2 Bit ordering

All vectors are most-significant-bit-on-the-left. Bit [0] is the LSB.

I.III.3 Numerical formats

Hexadecimal values are written `0x1A3A5C`. Binary values use a `0b` prefix. Two's-complement signed arithmetic is the default.

I.III.4 Hierarchy

The DSU is presented as a nested hierarchy:

DSU top → MAC cluster → single MAC → multiplier / adder

Each chapter zooms one level deeper than the previous.

I.III.5 Gate counts

Pre-synthesis estimates at the 28 nm target. Final numbers are updated after Genus synthesis closure.

I.IV Reference Documents

- GARUDA SoC Technical Reference Manual v2.0, Team AeroSoC, 2026.
- GARUDA SoC Deep Technical Reference v2.0, Team AeroSoC, 2026.
- The RISC-V Instruction Set Manual, Volume I: User-Level ISA, version 20191213.
- The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, version 20211203.

BLOCK II

DSU Top Module

II.I Block as Black Box

The DSU is a single sealed unit that sits inside the CPU's execute stage. Every cycle, it inspects the instruction the CPU is executing. If that instruction is one of the ten Custom-0 DSU instructions defined in Chapter IX, the DSU performs the corresponding operation. Otherwise it holds all internal state and consumes minimal switching power.

The DSU exposes its complete interface through 10 input signals and 8 output signals. Figure II.1 shows the boundary view.

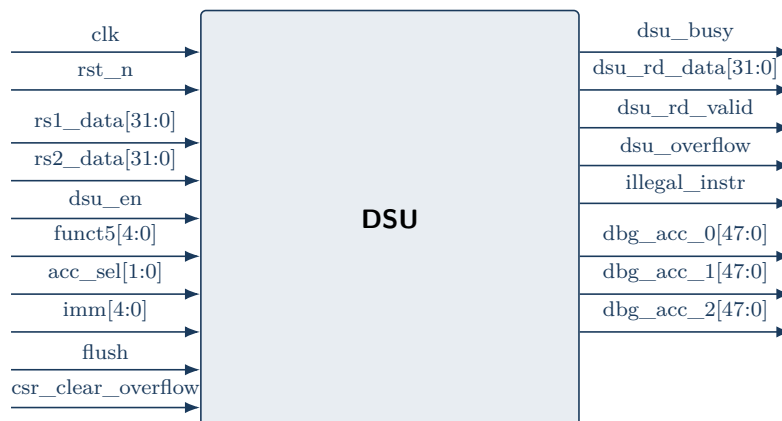


Figure II.1: DSU top module — external pin view.

II.II Input Pins

The 10 inputs split into three functional groups: infrastructure, operand data, and control. Table II.1 gives the full pin list.

II.II.1 Infrastructure signals

clk is the 200 MHz core clock. It drives every flip-flop inside the DSU. The DSU operates in a single clock domain — there are no internal clock dividers or asynchronous boundaries.

rst_n is the active-low synchronous-deassert reset. When asserted it clears every internal register: the three accumulators to zero, the stall FSM to IDLE, the overflow sticky flag to zero, and the product registers to don't-care (no reset wire saves area).

Table II.1: DSU top-module input signals.

Signal	Width	Source	Description
clk	1	Clock Divider	200 MHz core clock. Single domain.
rst_n	1	Reset Controller	Active-low; clears accumulators, FSM, overflow flag.
rs1_data[31:0]	32	Register File	Operand A (post-forwarding).
rs2_data[31:0]	32	Register File	Operand B (post-forwarding).
dsu_en	1	Core Decoder	Master gate; high for Custom-0 opcode.
funct5[4:0]	5	Core Decoder	Selects one of 10 DSU instructions.
acc_sel[1:0]	2	Core Decoder	Accumulator select (Fx / Fy / Mag / broadcast).
imm[4:0]	5	Core Decoder	Shift amount for MACSHIFT.
flush	1	Hazard Unit	Squash in-flight DSU op.
csr_clear_overflow	1	CSR Write Logic	Clears the sticky overflow flag.

II.II.2 Operand buses

`rs1_data[31:0]` and `rs2_data[31:0]` are the post-forwarding 32-bit operand values from the register file. They are present on the wires every cycle, regardless of whether the current instruction is a DSU instruction. The DSU consumes them only when `dsu_en` is high.

The full 32 bits are routed because MACDOT uses both halfwords. Other instructions use only `rs1_data[15:0]` and `rs2_data[15:0]`.

II.II.3 Control bundle

`dsu_en` is the master gate. It is high when bits [6:0] of the current instruction equal 0x0B (Custom-0 opcode). When low, all other control signals are don't-care and the DSU holds state.

`funct5[4:0]` selects one of the 10 DSU instructions per the encoding in Section IX.II. The 22 reserved codepoints trap as illegal-instruction.

`acc_sel[1:0]` selects which of the three accumulators the instruction targets: 00 is ACC_FX, 01 is ACC_FY, 10 is ACC_MAG, and 11 is broadcast (legal only with MACCLEAR).

`imm[4:0]` carries the shift amount for MACSHIFT. It is don't-care for all other instructions.

`flush` is asserted by the hazard unit when a younger trap or branch mispredict squashes the in-flight DSU operation. Accumulator writes are blocked for the flush cycle.

`csr_clear_overflow` is a single-cycle pulse from the CSR write logic, generated when firmware executes a CSR-write targeting the DSU overflow-clear address. It clears the sticky overflow flag.

II.III Output Pins

The 8 outputs split into pipeline control, writeback path, debug observation, and illegal-instruction trap. Table II.2 gives the full list.

Table II.2: DSU top-module output signals.

Signal	Width	Destination	Description
<code>dsu_busy</code>	1	Hazard Unit	Stall request; high during cycle 2 of 2-cycle ops.
<code>dsu_rd_data[31:0]</code>	32	Writeback Mux	MACREAD / MACSHIFT result.
<code>dsu_rd_valid</code>	1	Writeback Mux	Selects DSU result over ALU path.
<code>dsu_overflow</code>	1	CSR File	Sticky saturation flag.
<code>illegal_instr</code>	1	Trap Controller	Reserved encoding or illegal <code>acc_sel</code> detected.
<code>dsu_dbg_acc_0[47:0]</code>	48	Debug Module	ACC_FX tap, always live.
<code>dsu_dbg_acc_1[47:0]</code>	48	Debug Module	ACC_FY tap, always live.
<code>dsu_dbg_acc_2[47:0]</code>	48	Debug Module	ACC_MAG tap, always live.

II.III.1 Pipeline control

`dsu_busy` is the stall request to the hazard unit. It is high during the second cycle of any 2-cycle DSU instruction (`MAC_SEL`, `MACSUB`, `MACDOT`, `MACABS`). When high, the hazard unit freezes IF and ID for one bubble.

II.III.2 Writeback path

`dsu_rd_data[31:0]` carries the value written to the destination GPR for the three instructions that write a register: `MACREAD_LO`, `MACREAD_HI`, and `MACSHIFT`.

`dsu_rd_valid` tells the writeback mux to select `dsu_rd_data` as the destination value. When low, the writeback mux uses the ALU or load-data path instead.

`dsu_overflow` is a sticky flag. It is set when `MACSAT` clamps a value and remains high until cleared by `csr_clear_overflow` or `rst_n`.

II.III.3 Debug observation

`dsu_dbg_acc_0`, `dsu_dbg_acc_1`, `dsu_dbg_acc_2` are three 48-bit buses that expose each accumulator's current value continuously. They allow the debug module to inspect accumulator state in a halted core without injecting `MACREAD` instructions.

II.III.4 Illegal-instruction trap

`illegal_instr` is raised when the decoder detects a reserved `funct5` value or an `acc_sel=11` combination with any instruction other than `MACCLEAR`. It connects to the CPU's synchronous-exception input.

II.IV Module Pin Summary

The full port list, in the order it appears in the RTL module declaration, is captured in the cluster boundary table. The DSU's port boundary totals approximately 100 input bits and 180 output bits, with the three 48-bit debug taps dominating the output count.

BLOCK III

DSU Internal Architecture

III.1 Block Inventory

The DSU contains nine named blocks plus a small amount of glue logic. Figure III.1 shows their arrangement.

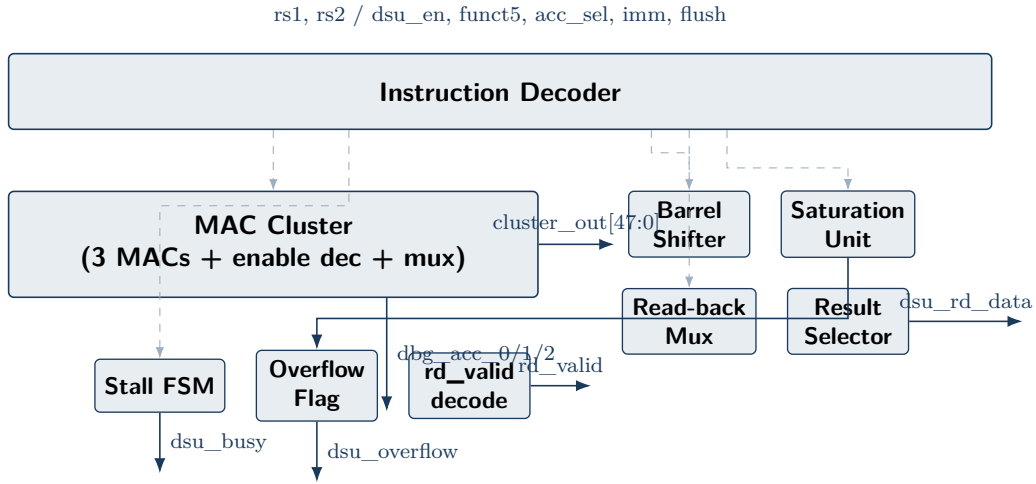


Figure III.1: DSU internal block diagram. Solid arrows are data buses, dashed arrows are control signals.

III.1.1 The nine blocks

1. **Instruction decoder** — combinational; produces all internal control from `funct5`, `acc_sel`, `dsu_en`; raises `illegal_instr`.
2. **MAC cluster** — three MAC instances plus enable decoder, saturation writeback router, and output mux. Detailed in Chapter IV.
3. **Barrel shifter** — 5-layer mux network performing arithmetic right-shift of the cluster output by an immediate.
4. **Saturation unit** — detects 48-to-32-bit overflow and clamps; produces both the write-back value and an overflow-detected pulse.
5. **Read-back mux** — selects the low or high half of the cluster output for transfer to a GPR.
6. **Result selector mux** — 32-bit 4:1 mux on the writeback return path.

7. **Stall FSM** — 2-state machine generating `dsu_busy`.
8. **Overflow sticky flag** — 1-bit register, set by saturation, cleared by CSR write.
9. **`dsu_rd_valid` decode** — 3-input OR gate combining the `aux_op` codepoints that write a GPR.

III.I.2 Glue logic

- Three flush-gating AND gates at the cluster boundary, one per MAC, on the accumulator write enables.
- Three per-MAC enable AND gates combining `cluster_en` with the cluster's internal per-MAC enables.
- Optional clock-gate cell on the aux-path inputs (synthesis-tool discretion).

III.I.3 What does not exist inside the DSU

The following are explicitly absent and must not be added without revising this specification:

- No memory of any kind beyond the accumulator and product registers within the cluster.
- No bus port (no AHB master, no AHB slave, no APB slave).
- No interrupt output. Saturation is signalled by the sticky flag; illegal-instruction is a synchronous exception.
- No internal CSR registers. The overflow flag is exposed as a one-bit input/output pair to the CPU's CSR file.
- No clock division or PLL.
- No SRAM macros or analogue cells.

III.II Signal Flow

Following the decoder's outputs to their consumers establishes the data flow through the DSU. Table III.1 enumerates every control signal generated by the decoder and its destination.

Table III.1: Decoder output signals and their destinations.

Signal	Width	Destination	Purpose
<code>cluster_en</code>	1	MAC Cluster	Master enable into cluster; gates per-MAC enables.
<code>add_sub</code>	1	MAC Cluster	Add (0) or subtract (1) at the adder.
<code>clear</code>	1	MAC Cluster	MACCLEAR pulse; routes via enable decoder.
<code>load</code>	1	MAC Cluster	MACLOAD select for accumulator-input mux.
<code>abs_en</code>	1	MAC Cluster	Engage abs pre-stage during MACABS.
<code>dot_en</code>	1	MAC Cluster	Engage MACDOT mode (both multipliers).
<code>aux_op[2:0]</code>	3	Aux Path	Selects shifter / saturation / read-back / none.
<code>is_2cycle</code>	1	Stall FSM	Marks 2-cycle instructions.
<code>illegal_instr</code>	1	(DSU output)	Synchronous exception.

III.II.1 Input-side paths

clk and **rst_n** fan out to the cluster, the stall FSM, and the overflow flag — the only stateful elements outside the cluster.

rs1_data and **rs2_data** go directly to the cluster. They never reach the decoder; control and data are strictly separated.

dsu_en, **funct5**, **acc_sel** land at the decoder. The decoder produces:

- **cluster_en**, **add_sub**, **clear**, **load**, **abs_en**, **dot_en** — to the cluster.
- **aux_op[2:0]** — to the barrel shifter, saturation unit, read-back mux, and result selector.
- **is_2cycle** — to the stall FSM.
- **illegal_instr** — directly out of the DSU to the trap controller.

imm[4:0] bypasses the decoder and feeds the barrel shifter directly.

flush reaches the stall FSM and the cluster's flush-gating ANDs. The cluster path is direct (one AND-gate delay) so that an in-flight 2-cycle MAC can be squashed reliably.

csr_clear_overflow goes only to the overflow sticky flag.

III.II.2 Output-side paths

dsu_busy is driven directly by the stall FSM.

dsu_rd_data is driven by the result selector mux. The mux selects between:

- Barrel shifter output (32 LSBs of the shifted 48-bit accumulator).
- Read-back mux output (accumulator LO or HI half).
- Zero (default — when no instruction targets a GPR).

dsu_rd_valid is the OR of **aux_op** codepoints 001 (shift), 011 (read-LO), and 100 (read-HI).

dsu_overflow is driven by the sticky flag. The flag's set-input comes from the saturation unit's **sat_overflow_detected** output.

dsu_dbg_acc_0/1/2 are direct wires from each MAC's accumulator output, bypassing the cluster's output mux.

illegal_instr is driven directly by the decoder.

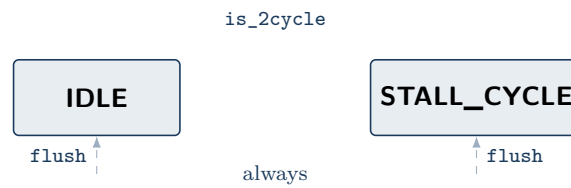
III.III The Aux Path

The barrel shifter, saturation unit, and read-back mux all read from the same `cluster_out` bus and all run combinationally every cycle. The result selector mux picks which output reaches `dsu_rd_data`. The saturation unit's output additionally writes back into the cluster — it is the only aux unit that modifies accumulator state.

This parallel-comb organisation means the aux path adds no extra cycles to instruction execution. `MACSHIFT`, `MACSAT`, `MACREAD_LO`, and `MACREAD_HI` all complete in one cycle.

III.IV Stall FSM Behaviour

The stall FSM has two states, encoded in a single flip-flop:



Transitions:

- **IDLE** → **STALL_CYCLE** when `is_2cycle` is asserted and `flush` is low.
- **STALL_CYCLE** → **IDLE** unconditionally on the next clock edge.
- Any state → **IDLE** if `flush` is asserted.

`dsu_busy` is high if and only if the state is **STALL_CYCLE**.

BLOCK IV

MAC Cluster

IV.I Purpose

The cluster instantiates three identical MAC units and provides per-MAC enable decoding, saturation writeback routing, and output multiplexing. It is a thin wrapper around the three MAC instances — the bulk of the cluster’s gate count lives inside the MACs themselves.

The three MACs map onto the APF repulsion-vector computation:

- **MAC 0 / ACC_FX** — holds the running sum of X-axis repulsion contributions across all neighbour drones.
- **MAC 1 / ACC_FY** — holds the running sum of Y-axis repulsion contributions.
- **MAC 2 / ACC_MAG** — scratch accumulator for distance and magnitude math during APF, reused by EKF for matrix-multiply intermediates.

The three MACs are accumulator contexts, not parallel compute lanes. Only one MAC is enabled per cycle, selected by `acc_sel`. Parallel issue (a hypothetical `MAC_ALL`) is a deferred enhancement not part of this revision.

IV.II Cluster Architecture

Figure IV.1 shows the cluster’s internal arrangement.

IV.II.1 Block inventory

1. **MAC 0** — owning `ACC_FX`. Internal structure in Chapter V.
2. **MAC 1** — owning `ACC_FY`.
3. **MAC 2** — owning `ACC_MAG`.
4. **Per-MAC enable decoder** — combinational; takes `cluster_en`, `acc_sel`, `clear`, `flush`; outputs three per-MAC `en` and `clr` signals.
5. **Saturation writeback router** — combinational; routes `sat_writeback[47:0]` to the MAC indexed by `acc_sel` when `sat_writeback_en` is high.
6. **Output mux** — 48-bit 3:1 mux selecting one of the three accumulator outputs based on `acc_sel`.

Always-live debug taps are direct wires from each MAC’s accumulator output to the cluster output port, bypassing the mux.

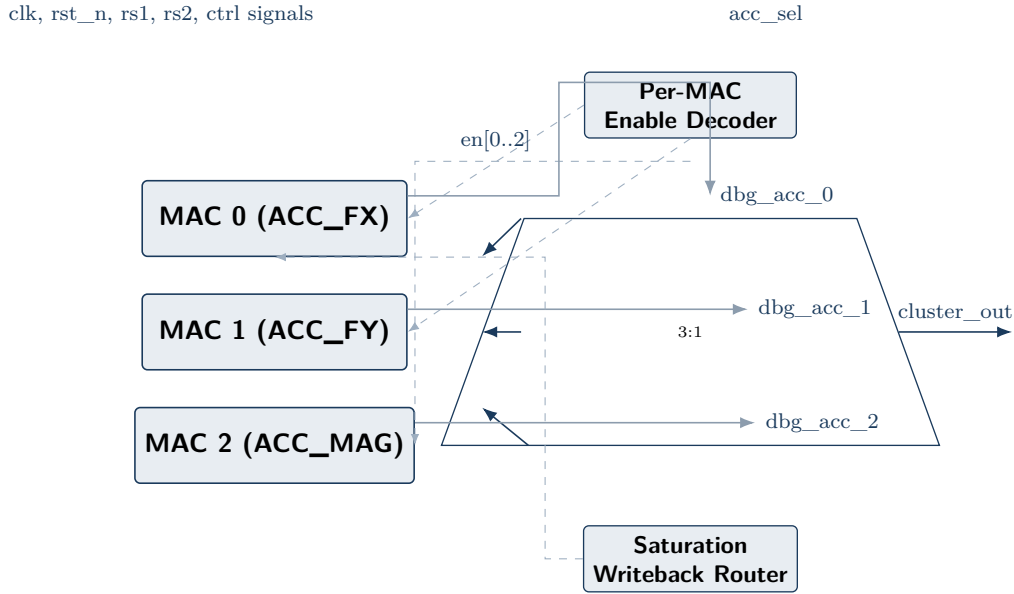


Figure IV.1: MAC cluster internal architecture. Lighter arrows are the always-live debug taps; they bypass the output mux.

IV.III Cluster Pin Contract

IV.IV Per-MAC Enable Decoder

The enable decoder is the small piece of logic that turns `acc_sel` into three one-hot per-MAC enables. Its truth table is given in Table IV.2.

The only legal use of `acc_sel=11` is broadcast MACCLEAR. All other instructions with `acc_sel=11` raise `illegal_instr` in the upstream decoder before they reach the cluster.

IV.V Saturation Writeback Routing

MACSAT is the only aux instruction that modifies accumulator state. The DSU-level saturation unit produces a clamped 48-bit value on `sat_writeback`; the cluster's saturation router directs that value to the accumulator-input port of the MAC indexed by `acc_sel`.

The router is implemented as three 48-bit AND-OR multiplexers, one per MAC. Each MAC's accumulator-input mux (Section V.VIII) selects between adder output, zero, sign-extended `rs1`, and `sat_writeback`; the saturation router gates the latter input so that only the addressed MAC sees a non-zero `sat_writeback` value during MACSAT.

IV.VI Signal Flow Inside the Cluster

IV.VI.1 Broadcast signals

The following signals enter the cluster and fan out unchanged to all three MAC instances. Each MAC accepts them as inputs; only the enabled MAC acts.

- `clk`, `rst_n`
- `rs1_data[31:0]`, `rs2_data[31:0]`
- `add_sub`, `abs_en`, `dot_en`, `load`

Table IV.1: MAC cluster external pin contract.

Signal	Dir	Width	Function
<code>clk</code>	in	1	Clock to all three MACs.
<code>rst_n</code>	in	1	Reset to all three accumulators.
<code>rs1_data</code>	in	32	Operand A; broadcast to all MACs.
<code>rs2_data</code>	in	32	Operand B; broadcast to all MACs.
<code>cluster_en</code>	in	1	Cluster-level enable.
<code>acc_sel</code>	in	2	Accumulator select.
<code>add_sub</code>	in	1	Add/subtract control.
<code>clear</code>	in	1	Clear pulse.
<code>abs_en</code>	in	1	Abs pre-stage enable.
<code>dot_en</code>	in	1	MACDOT mode select.
<code>load</code>	in	1	MACLOAD select.
<code>flush</code>	in	1	Squash in-flight write.
<code>sat_writeback</code>	in	48	Clamped value from saturation unit.
<code>sat_writeback_en</code>	in	1	Routes <code>sat_writeback</code> to addressed MAC.
<code>cluster_out</code>	out	48	Muxed accumulator output (selected by <code>acc_sel</code>).
<code>dbg_acc_0</code>	out	48	ACC_FX tap, always live.
<code>dbg_acc_1</code>	out	48	ACC_FY tap, always live.
<code>dbg_acc_2</code>	out	48	ACC_MAG tap, always live.

IV.VI.2 Per-MAC signals

These are generated inside the cluster by the enable decoder or the saturation router and connect to one MAC each:

- `en[0]`, `en[1]`, `en[2]` — per-MAC enables.
- `clr[0]`, `clr[1]`, `clr[2]` — per-MAC clear pulses.
- Saturation-writeback enables (per MAC, gated by `acc_sel` and `sat_writeback_en`).

`flush` enters the cluster and reaches both the enable decoder (forcing all enables low) and each MAC's accumulator write-enable gate (blocking the in-flight write directly).

IV.VI.3 Outputs

The three MAC `mac_out[47:0]` buses leave the cluster on two paths simultaneously:

- Through the 3:1 output mux to `cluster_out[47:0]` (selected by `acc_sel`).
- Directly out to `dbg_acc_0/1/2[47:0]` as always-live taps.

IV.VII Cluster Gate Count

The cluster accounts for approximately 96 % of the DSU's total gate count. Within the cluster, the six 16×16 multipliers account for roughly 70 % of the area.

Table IV.2: Per-MAC enable decoder truth table.

Operation	acc_sel	en[0]	en[1]	en[2]	clr[0]	clr[1]	clr[2]
Targeted MAC op	00	1	0	0	0	0	0
Targeted MAC op	01	0	1	0	0	0	0
Targeted MAC op	10	0	0	1	0	0	0
MACCLEAR broadcast	11	1	1	1	1	1	1
Non-DSU instr	—	0	0	0	0	0	0
Flush asserted	any	0	0	0	0	0	0

Table IV.3: MAC cluster gate count breakdown (pre-synthesis estimates).

Component	Gates
Six 16×16 multipliers (3 MACs × 2 each)	~16,800
Three 48-bit Kogge-Stone adders (with CSA pre-stage)	~3,150
Three accumulator registers (48 flops each + clock-gate)	~3,000
Three product registers (33+33 flops each, clock-gated)	~4,200
Three abs pre-stages (per-MAC, both operands)	~300
Three operand routers + operand isolation	~270
Three 3:2 compressors (for MACDOT)	~450
Three accumulator-write muxes	~600
Per-MAC enable decoder + saturation router + output mux	~250
Cluster total	~29,020

BLOCK V

Single MAC Unit

V.I Function

One MAC unit computes:

$$\text{acc} \leftarrow \text{acc} \pm (\text{rs1}[15:0] \times \text{rs2}[15:0])$$

or, in MACDOT mode:

$$\text{acc} \leftarrow \text{acc} + (\text{rs1}[31:16] \times \text{rs2}[31:16]) + (\text{rs1}[15:0] \times \text{rs2}[15:0])$$

The accumulator is a 48-bit signed register that retains its value across all non-DSU instructions and across DSU instructions targeting other accumulators. It is cleared by MACCLEAR and replaced by clamped values during MACSAT, or loaded directly via MACLOAD.

V.II Pin Contract

Table V.1: Single MAC unit pin contract.

Signal	Dir	Width	Function
clk	in	1	Clock to product reg and accumulator reg.
rst_n	in	1	Resets accumulator to zero.
rs1	in	32	Operand A; full 32 bits for MACDOT.
rs2	in	32	Operand B; full 32 bits for MACDOT.
en	in	1	This MAC's enable from cluster decoder.
add_sub	in	1	Add (0) or subtract (1).
clear	in	1	Clear accumulator to zero.
load	in	1	MACLOAD select.
abs_en	in	1	Engage absolute-value pre-stage.
dot_en	in	1	Engage MACDOT mode.
flush	in	1	Squash in-flight accumulator write.
sat_writeback	in	48	Clamped value for MACSAT.
sat_writeback_en	in	1	Selects sat_writeback in acc-input mux.
mac_out	out	48	Accumulator value, always live.

V.III Internal Architecture

Figure V.1 shows the MAC's datapath under MACDOT Option A (two multipliers per MAC, supporting real 2-cycle MACDOT).

rs1[31:0], rs2[31:0]

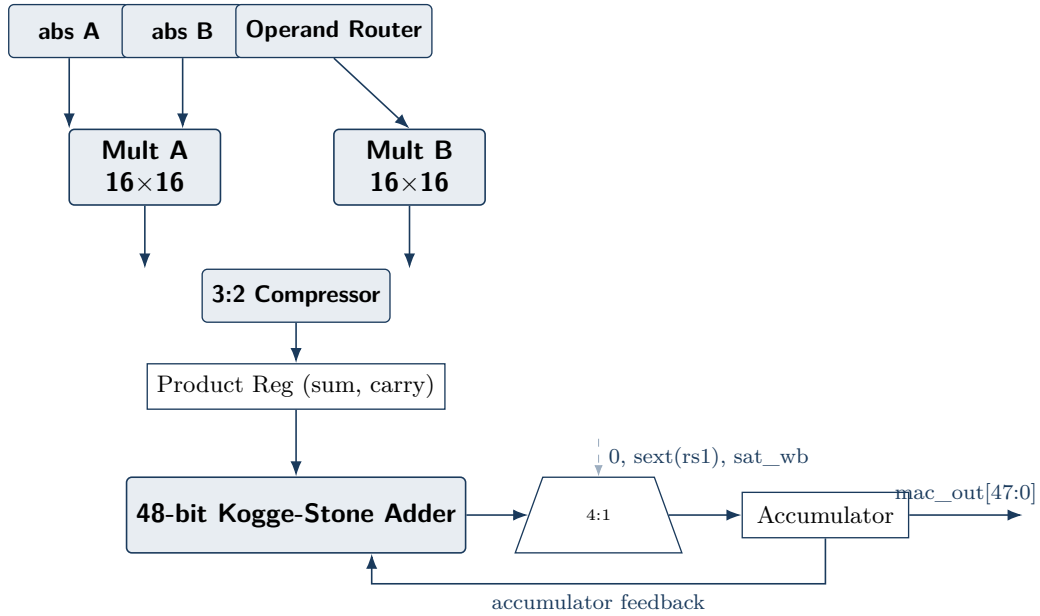


Figure V.1: Single MAC internal datapath under MACDOT Option A. Two multipliers feed a 3:2 compressor that produces a carry-save pair; the product register holds both during cycle 1; the 48-bit adder produces the new accumulator value in cycle 2.

V.III.1 Block inventory

1. **Absolute-value pre-stage A** — conditionally inverts `rs1[15:0]` based on its sign bit when `abs_en` is high.
2. **Absolute-value pre-stage B** — identical, for `rs2[15:0]`.
3. **Operand router** — selects which 16-bit halves of `rs1` and `rs2` feed each multiplier in MACDOT vs normal mode.
4. **Multiplier A** — 16×16 signed Booth Radix-4 + Dadda + Kogge-Stone. Active in all multiply-using modes.
5. **Multiplier B** — identical to Multiplier A; active only in MACDOT.
6. **3:2 compressor** — merges two 32-bit products into a carry-save pair during MACDOT; bypassed otherwise.
7. **Product register** — two 33-bit flop banks (sum and carry words) latched at the end of cycle 1.
8. **48-bit accumulator adder** — Kogge-Stone parallel-prefix; covered in Chapter VII.
9. **Accumulator-write input mux** — 4:1 mux selecting between adder output, zero, sign-extended `rs1`, and `sat_writeback`.

10. **Accumulator register** — 48 flip-flops, clock-gated by $\text{en} \wedge \neg \text{flush}$.

V.IV Absolute-Value Pre-Stages

Each MAC owns its own pair of abs pre-stages (one per operand). When `abs_en` is low, the pre-stage passes its operand through unchanged. When high, it XORs every bit with the sign bit (`operand[15]`), then adds 1 via the multiplier’s carry-in network.

This handles two’s-complement negation in a single gate-level XOR row plus a carry adjustment, avoiding a full adder stage in front of the multiplier.

- Cost: 16 XOR gates per operand $\times$ 2 operands = 32 XOR gates per MAC.
- Three MACs: ~ 96 XOR gates plus the carry-injection logic.

V.V Operand Router

The router selects which 16-bit halves of `rs1_data` and `rs2_data` reach each multiplier:

Table V.2: Operand router truth table.

<code>dot_en</code>	Mult A input 1	Mult A input 2	Mult B input 1	Mult B input 2
0	<code>abs(rs1[15:0])</code>	<code>abs(rs2[15:0])</code>	0 (isolated)	0 (isolated)
1	<code>rs1[15:0]</code>	<code>rs2[15:0]</code>	<code>rs1[31:16]</code>	<code>rs2[31:16]</code>

When `dot_en` is low, Multiplier B’s inputs are driven to zero by operand-isolation AND gates, preventing it from burning switching power.

V.VI Two Multipliers and the 3:2 Compressor

Both multipliers produce 32-bit signed products combinationally. In normal mode, only Multiplier A’s product is meaningful; Multiplier B drives zero. In MACDOT mode, both produce real products and must be combined.

The 3:2 compressor reduces the two 32-bit products to a carry-save pair (a sum word and a carry word) using one row of full-adders. This row adds approximately 0.2 ns to the cycle-1 critical path but avoids needing a full carry-propagate adder, which would be considerably slower.

When `dot_en` is low, the compressor’s carry word output is forced to zero, so the cycle-2 adder behaves as a normal two-input add.

V.VII Product Register

The product register has two banks: a 33-bit sum register and a 33-bit carry register (one bit wider than the 32-bit product to accommodate the compressor output). Both banks share a clock-gate driven by the MAC’s `en` signal.

When the MAC is not enabled, neither bank latches, saving dynamic power. The values held are don’t-care — the next enabling cycle overwrites them.

V.VIII Accumulator-Write Input Mux

The 4:1 mux at the accumulator’s data input selects between the four sources of accumulator updates:

Table V.3: Accumulator-input mux selection.

Mux input	Source	Selected when	Used by
00	48-bit adder output	default	MAC_SEL, MACSUB, MACDOT, MACABS
01	Zero	clear=1	MACCLEAR
10	Sign-extended rs1	load=1	MACLOAD
11	sat_writeback	sat_writeback_en=1	MACSAT

The select signals are decoded internally from the broadcast control signals `clear`, `load`, and `sat_writeback_en`. At most one of these is high at a time; the mux defaults to the adder output when all are low.

V.IX Accumulator Register

The accumulator is 48 flip-flops with a clock-gate cell driven by:

$$\text{actual_write_en} = \text{en} \wedge \neg \text{flush}$$

The accumulator resets to zero on `rst_n` assertion. It updates on the rising clock edge whenever `actual_write_en` is high. Its output is exposed continuously on `mac_out` — the always-live output bus consumed by the cluster’s output mux, the cluster’s debug taps, the DSU’s aux path, and (via the adder feedback path) its own input.

V.X Cycle-by-Cycle Behaviour

MACREAD_LO, MACREAD_HI, and MACSHIFT do not engage any internal MAC logic. They consume `mac_out` externally through the cluster output mux and the DSU aux path.

V.XI Flush Handling

When `flush` is asserted in any cycle, the accumulator write enable for that cycle is forced to zero. The accumulator holds its previous value. In-flight 2-cycle MACs are discarded silently — no architectural state changes.

The product register’s clock-gate is driven by `en` alone (without flush gating), because the product register’s contents are don’t-care unless they reach the accumulator. Suppressing the accumulator write is sufficient to make the squashed instruction architecturally invisible.

V.XII Power Management

Each MAC is independently clock-gated. When this MAC’s `en` is low:

- Product register holds, no switching.
- Accumulator register holds, no switching.
- Multiplier inputs are quiet (no operand changes from upstream broadcast), so no combinatorial switching.
- Multiplier B is additionally operand-isolated by `dot_en`.

In typical operation only one of the three MACs is active per cycle, so two-thirds of the cluster’s switching power is gated. During non-DSU instructions, all three MACs are gated.

Table V.4: Per-instruction cycle-by-cycle behaviour inside one MAC.

Instruction	Cycle 1	Cycle 2
MAC_SEL	Mult A produces product; compressor passes through; product reg latches.	Adder computes $acc + sext(product)$; acc latches.
MACSUB	Same as MAC_SEL.	Adder computes $acc - sext(product)$; acc latches.
MACABS	Abs pre-stage active; mult A produces $ rs1 \times rs2 $; product reg latches.	Same as MAC_SEL cycle 2.
MACDOT	Both mults active; compressor produces carry-save pair; product reg latches both words.	Adder computes $acc + sext(sum) + sext(carry)$ via CSA + Kogge-Stone; acc latches.
MACCLEAR	Acc-input mux selects zero; acc latches zero. (Multiplier idle.)	—
MACLOAD	Acc-input mux selects $sext(rs1)$; acc latches. (Multiplier idle.)	—
MACSAT	Acc-input mux selects sat_writeback ; acc latches clamped value.	—
MACREAD_LO	MAC's en =0; nothing happens internally; mac_out consumed externally.	—
MACREAD_HI	Same as MACREAD_LO.	—
MACSHIFT	Same as MACREAD_LO; barrel shifter reads mac_out .	—

BLOCK VI

The 16×16 Signed Multiplier

VI.I Architecture Choice

The multiplier uses three stacked optimisations, each addressing a different bottleneck:

- **Modified Booth Radix-4** encoding for partial product generation, reducing the partial-product count from 16 to 9 rows.
- **Dadda compression tree** for accumulating the 9 partial products into a carry-save pair using the minimum adder cell count at a given critical-path depth.
- **Kogge-Stone parallel-prefix adder** for the final carry-propagate addition, achieving flat $\log_2 N$ delay with uniform fanout.

The combined delay at 28 nm is approximately 1.5 ns, comfortably inside the 5 ns cycle budget at 200 MHz.

VI.II Pin Contract

Table VI.1: 16×16 signed multiplier pin contract.

Signal	Dir	Width	Function
a	in	16	Signed multiplicand.
b	in	16	Signed multiplier (Booth-encoded internally).
p	out	32	Signed product.

VI.III Stage 1 — Booth Radix-4 Encoder

Booth Radix-4 examines three adjacent bits of the multiplier (**b**) at a time and encodes them into a Booth digit selecting one of $\{0, \pm a, \pm 2a\}$. Nine encoder cells produce nine Booth digits, each represented as three control signals: **neg**, **one**, **two**.

The implicit-zero rule places a zero bit to the right of **b**[0] when encoding the bottom digit. The top digit treats **b**[16] as a sign-extension of **b**[15].

Each encoder cell is approximately 5 gates. Nine cells total ≈ 45 gates.

VI.IV Stage 2 — Partial Product Generation

Each Booth digit drives a partial product generator that produces a 17-bit signed value: $\{0, +a, -a, +2a, -2a\}$ depending on the digit. The $\pm 2a$ option is implemented as a left-shift by

Table VI.2: Booth Radix-4 encoding table.

$b[i+1]$	$b[i]$	$b[i-1]$	Booth digit	Partial product	neg	one	two
0	0	0	0	0	0	0	0
0	0	1	+1	$+a$	0	1	0
0	1	0	+1	$+a$	0	1	0
0	1	1	+2	$+2a$	0	0	1
1	0	0	-2	$-2a$	1	0	1
1	0	1	-1	$-a$	1	1	0
1	1	0	-1	$-a$	1	1	0
1	1	1	0	0	0	0	0

one with a zero injected at the bottom; the $\pm$ sign is implemented as a conditional XOR row driven by **neg**, with the +1 correction injected as a carry-in into the column to the right during compression.

The nine partial products are positioned at column offsets 0, 2, 4, $\dots$, 16 (two bits per Booth digit). This creates the triangular grid the compression tree processes.

Approximate gate count: 6 gates per bit $\times$ 17 bits $\times$ 9 cells = 918 gates of muxing, plus 2 gates per bit for negation XOR = 1,224 gates total. Practically rounds to $\sim$ 1,200 gates.

VI.V Stage 3 — Dadda Compression Tree

Dadda compression reduces the 9-deep partial-product grid to two rows (a sum row and a carry row) using half-adders (2:2 reduction) and full-adders (3:2 reduction). The strategy is the minimum-cell variant: at each level, use only as many adders as are necessary to bring every column down to the target height, where the target heights for nine rows are $9 \rightarrow 6 \rightarrow 4 \rightarrow 3 \rightarrow 2$.

Table VI.3: Dadda compression tree levels for nine partial products.

Level	Input max height	Target height	Half-adders	Full-adders
1	9	6	few	$\sim$ 25
2	6	4	few	$\sim$ 20
3	4	3	few	$\sim$ 15
4	3	2 (mostly)	few	$\sim$ 10
5	≤ 3	2 (final)	$\sim$ 5	$\sim$ 5
Totals			$\sim$15	$\sim$75

Total: approximately 75 full-adders ($\sim$ 9 gates each) and 15 half-adders ($\sim$ 3 gates each), giving $\sim$ 700 gates.

VI.VI Stage 4 — Final 32-bit Kogge-Stone Adder

The compression tree produces two 32-bit numbers in carry-save form. The final adder converts them to a single 32-bit signed product using a Kogge-Stone parallel-prefix addition (5 levels for 32 bits).

The Kogge-Stone structure is described in detail for the 48-bit accumulator adder in Chapter VII; the 32-bit final-add inside the multiplier uses the identical architecture with one fewer level.

Table VI.4: 16×16 multiplier gate count breakdown.

Stage	Gates
Booth encoders (9 cells, ~5 gates each)	~45
Partial product generator (9 cells, 17-bit each)	~1,200
Dadda compression tree (75 FA + 15 HA)	~700
Kogge-Stone final 32-bit adder	~850
Total per multiplier	~2,800

VI.VII Timing

The combinational delay through the multiplier breaks down approximately as:

Stage	Delay (28 nm)
Booth encoding	0.10 ns
Partial product generation	0.15 ns
Dadda compression (5 levels)	0.75 ns
Kogge-Stone final adder (5 levels)	0.50 ns
Total	~1.50 ns

The compression tree dominates the delay because each level adds approximately one full-adder delay. The final adder is comparatively fast because Kogge-Stone’s parallel-prefix structure is shallower per bit than ripple-style addition.

VI.VIII Multiplier Instances in the DSU

- Two multipliers per MAC (Multiplier A and Multiplier B for MACDOT).
- Three MACs.
- Total of six multiplier instances in the cluster.

The six multipliers consume approximately $6 \times 2,800 = 16,800$ gates, or roughly 70 % of the cluster’s gate count.

BLOCK VII

The 48-bit Accumulator Adder

VII.I Function

One 48-bit Kogge-Stone parallel-prefix adder lives inside each MAC. It executes cycle 2 of every multiply-accumulate instruction:

$$\text{acc}_{\text{new}} = \text{acc}_{\text{current}} + \text{sign_extend}_{48}(\text{product})$$

for MAC_SEL, MACABS; with subtract for MACSUB; with three-input add (accumulator + sum-word + carry-word) for MACDOT.

This adder is the cycle-2 critical path of the MAC. Its timing determines the maximum operating frequency of the DSU.

VII.II Pin Contract

Table VII.1: 48-bit Kogge-Stone accumulator-adder pin contract.

Signal	Dir	Width	Function
a	in	48	First operand (accumulator value).
b1	in	48	Second operand (sign-extended product or sum-word).
b2	in	48	Third operand (carry-word for MACDOT, zero otherwise).
cin	in	1	Carry-in; provides +1 for subtract.
sub_invert	in	1	XOR with b1 to invert for subtract.
dot_en	in	1	Engages CSA pre-stage for three-input add.
sum	out	48	Result.
cout	out	1	Carry-out (typically discarded).

VII.II.1 Two-input mode

For MAC_SEL / MACSUB / MACABS:

- a = accumulator current value.
- b1 = sign-extended product.
- b2 = zero (compressor's carry word forced to zero).
- sub_invert = 1 for MACSUB, 0 otherwise.
- cin = sub_invert (provides the +1 for two's-complement subtract).

VII.II.2 Three-input mode

For MACDOT: **b1** is the sign-extended sum word and **b2** is the sign-extended carry word, both from the product register. The adder's internal CSA pre-stage reduces (a, b1, b2) to two operands before the Kogge-Stone prefix network.

VII.III Parallel-Prefix Structure

A parallel-prefix adder computes the sum in three phases:

1. **Pre-processing** — generate per-bit **G[i]** (generate) and **P[i]** (propagate) signals.
2. **Prefix network** — combine **G** and **P** signals across spans of bits using the prefix operator $\bullet$, in a tree of $\log_2 N$ levels.
3. **Post-processing** — compute each sum bit from **P[i]** and the carry into bit i .

VII.III.1 Phase 1 — Pre-processing

For every bit position i from 0 to 47:

$$\begin{aligned} G[i] &= a[i] \wedge b[i] \\ P[i] &= a[i] \oplus b[i] \end{aligned}$$

48 AND gates and 48 XOR gates = approximately 150 gates.

When the subtract path is active, **b[i]** is first XORed with **sub_invert**, effectively inverting it. The **cin** signal injects the +1 correction at bit 0.

VII.III.2 Phase 2 — Prefix network

The prefix operator $\bullet$ combines two span representations into a wider span:

$$\begin{aligned} G_{i:k} &= G_{j+1:k} \vee (P_{j+1:k} \wedge G_{i:j}) \\ P_{i:k} &= P_{j+1:k} \wedge P_{i:j} \end{aligned}$$

This operator is associative, so spans can be combined in a tree. Kogge-Stone arranges the tree such that every cell has exactly fanout-2 and the tree height is $\lceil \log_2 N \rceil$. For 48 bits this is 6 levels.

Table VII.2: Kogge-Stone prefix network for 48-bit addition.

Level	Span	Cells in this level	Cumulative cells
1	2	47	47
2	4	46	93
3	8	44	137
4	16	40	177
5	32	32	209
6	48	16	225

Total prefix cells ≈ 225 across all levels. At approximately 3.5 gates per cell, the prefix network is ~ 800 gates.

VII.III.3 Phase 3 — Post-processing

The carry into bit i is the group-generate signal $G_{0:i-1}$, computed by the prefix network. The sum at bit i is:

$$\text{sum}[i] = P[i] \oplus G_{0:i-1}$$

48 XOR gates $\approx$ 50 gates.

VII.IV CSA Pre-Stage for MACDOT

When `dot_en` is active, the adder receives three inputs (a, b1, b2). A row of carry-save adders (3:2 compressors) reduces these to two inputs before the Kogge-Stone prefix. The CSA row adds one full-adder delay (~ 0.2 ns) to the critical path.

- 48 full-adders for the CSA row: ~ 430 gates.
- Bypass mux on the CSA inputs: ~ 50 gates.

Total CSA cost per adder $\approx$ 480 gates. Three adders: $\sim 1,440$ gates across the cluster.

VII.V Total Gate Count

Table VII.3: 48-bit accumulator-adder gate count.

Component	Gates
Pre-processing (G, P generation, 48 bits)	~ 150
Prefix network (6 levels, 225 cells, ~ 3.5 gates each)	~ 800
Post-processing (sum generation, 48 bits)	~ 50
CSA pre-stage (MACDOT mode, 48 full-adders + bypass)	~ 480
Subtract control (XOR row + carry-in)	~ 50
Total per adder	$\sim 1,530$

Three adders across the cluster: approximately 3,150 gates of accumulator-adder hardware. About 13% of the cluster's total gate count.

VII.VI Timing

Stage	Delay (28 nm)
CSA pre-stage (MACDOT only)	0.20 ns
Pre-processing (G, P generation)	0.15 ns
Prefix network (6 levels)	0.90 ns
Post-processing (sum generation)	0.15 ns
Total (two-input mode)	~ 1.20 ns
Total (MACDOT mode)	~ 1.40 ns

Both modes fit comfortably in the 5 ns cycle budget. The 48-bit adder is shallower than the multiplier despite being wider, because Kogge-Stone's depth is logarithmic in operand width while the multiplier's compression tree has fixed depth proportional to the number of partial products.

BLOCK VIII

DSU Auxiliary Blocks

This chapter covers the six smaller blocks inside the DSU but outside the cluster. None has internal hierarchy worth a separate chapter; each is described in a single section.

VIII.1 Barrel Shifter

VIII.1.1 Pin contract

Table VIII.1: Barrel shifter pin contract.

Signal	Dir	Width	Function
<code>acc_in</code>	in	48	48-bit accumulator value.
<code>shamt</code>	in	5	Shift amount (<code>imm[4:0]</code>).
<code>shifter_en</code>	in	1	Aux-path gating.
<code>shift_out</code>	out	32	Arithmetic-right-shifted low 32 bits.

VIII.1.2 Architecture

A 5-layer mux network performs arithmetic right-shift by any amount 0–31. Each layer shifts conditionally by a power of 2:

- Layer 0: shift right by 1 if `shamt[0]`, else pass.
- Layer 1: shift right by 2 if `shamt[1]`, else pass.
- Layer 2: shift right by 4 if `shamt[2]`, else pass.
- Layer 3: shift right by 8 if `shamt[3]`, else pass.
- Layer 4: shift right by 16 if `shamt[4]`, else pass.

Arithmetic right-shift fills the vacated top bits with copies of the sign bit. Each layer is implemented as 48 2:1 muxes per bit position; the input that gets “shifted into” the position is either the bit 2^k positions higher (when shift is active) or the bit itself (when shift is inactive).

VIII.1.3 Gate count

48 muxes $\times$ 5 layers $\times$ 3 gates each = 720 gates.

VIII.I.4 Timing

~1.0 ns at 28 nm (5 mux delays).

VIII.I.5 Output width

The output is 32 bits — the low 32 bits of the shifted 48-bit value. Bits 47–32 are discarded because the GPR destination is 32 bits wide.

VIII.II Saturation Unit

VIII.II.1 Pin contract

Table VIII.2: Saturation unit pin contract.

Signal	Dir	Width	Function
acc_in	in	48	48-bit accumulator value.
sat_en	in	1	Aux-path gating; high during MACSAT.
sat_out	out	48	Clamped value (writes back to accumulator).
sat_overflow_detected	out	1	Pulses high if clamping occurred.

VIII.II.2 Architecture

A 48-bit signed value fits in 32-bit signed range if and only if bits 47–31 are all equal (all zeros for positive, all ones for negative). The saturation unit detects this condition and clamps when it fails.

Detection logic

$$\begin{aligned}
 \text{pos_overflow} &= \neg \text{acc_in}[47] \wedge (\text{OR of acc_in}[46:31]) \\
 \text{neg_overflow} &= \text{acc_in}[47] \wedge (\text{AND-NOT of acc_in}[46:31]) \\
 \text{sat_overflow_detected} &= (\text{pos_overflow} \vee \text{neg_overflow}) \wedge \text{sat_en}
 \end{aligned}$$

Output mux

- If pos_overflow: output 0x0000_0000_7FFF_FFFF.
- If neg_overflow: output 0xFFFF_FFFF_8000_0000.
- Otherwise: output acc_in unchanged.

VIII.II.3 Gate count

~200 gates total: two 16-bit reductions plus a 48-bit 3:1 mux plus output gating.

VIII.III Read-back Multiplexer

VIII.III.1 Pin contract

VIII.III.2 Architecture

A 32-wide 2:1 mux:

Table VIII.3: Read-back mux pin contract.

Signal	Dir	Width	Function
acc_in	in	48	48-bit accumulator value.
hi_sel	in	1	0=read LO, 1=read HI.
readback_out	out	32	Selected half (sign-extended for HI).

- LO path: output bits 31–0 are `acc_in[31:0]`.
- HI path: output bits 15–0 are `acc_in[47:32]`; bits 31–16 are sign-extended copies of `acc_in[47]`.

Sign extension on the HI path is pure fanout — no gates.

VIII.III.3 Gate count

~100 gates (32 2:1 muxes at ~3 gates each).

VIII.IV Result Selector Mux

VIII.IV.1 Pin contract

Table VIII.4: Result selector mux pin contract.

Signal	Dir	Width	Function
shifter_out	in	32	From barrel shifter.
readback_out	in	32	From read-back mux.
aux_op	in	3	Mux select.
dsu_rd_data	out	32	Drives DSU output.

VIII.IV.2 Architecture

A 32-bit 3:1 mux selecting between barrel shifter output, read-back mux output, and zero. The select is decoded from `aux_op[2:0]`:

aux_op	Mux selection
001	Barrel shifter output
011	Read-back LO
100	Read-back HI (drives <code>hi_sel=1</code> to read-back mux)
Other	Zero

VIII.IV.3 Gate count

~150 gates.

VIII.V Stall FSM

VIII.V.1 Pin contract

Table VIII.5: Stall FSM pin contract.

Signal	Dir	Width	Function
clk	in	1	Clock to state flop.
rst_n	in	1	Resets state to IDLE.
is_2cycle	in	1	From decoder; triggers stall.
flush	in	1	Forces state to IDLE.
dsu_busy	out	1	High in STALL_CYCLE.

VIII.V.2 State machine

Two states encoded in one flip-flop:

- IDLE (state=0): dsu_busy=0.
- STALL_CYCLE (state=1): dsu_busy=1.

Next-state logic

```
always_comb begin
    if (flush)
        next_state = IDLE;
    else if (state == IDLE && is_2cycle)
        next_state = STALL_CYCLE;
    else
        next_state = IDLE;
end
```

VIII.V.3 Gate count

1 flip-flop + ~5 gates of next-state logic = ~15 gates total.

VIII.VI Overflow Sticky Flag

VIII.VI.1 Pin contract

Table VIII.6: Overflow sticky flag pin contract.

Signal	Dir	Width	Function
clk	in	1	Clock to flag flop.
rst_n	in	1	Resets flag to 0.
sat_overflow_detected	in	1	Sets flag on saturation.
csr_clear_overflow	in	1	Clears flag (priority over set).
dsu_overflow	out	1	Sticky flag value.

VIII.VI.2 Logic

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)                dsu_overflow <= 1'b0;
    else if (csr_clear_overflow) dsu_overflow <= 1'b0;
    else if (sat_overflow_detected) dsu_overflow <= 1'b1;
    // else: hold
end

```

Clear has priority over set, so a simultaneous CSR clear and MACSAT clamp results in a cleared flag. This case should not arise in well-formed firmware.

VIII.VI.3 Gate count

1 flip-flop + ~ 8 gates = ~ 10 gates total.

VIII.VII Summary

Table VIII.7: Auxiliary block gate-count summary.

Block	Gates	Type
Barrel shifter	~ 720	Combinational (5 mux layers)
Saturation unit	~ 200	Combinational (compare + clamp)
Read-back mux	~ 100	Combinational (2:1 mux + sign-ext)
Result selector mux	~ 150	Combinational (4:1 mux)
Stall FSM	~ 15	Sequential (1 flop)
Overflow sticky flag	~ 10	Sequential (1 flop)
Total	$\sim 1,195$	

The six auxiliary blocks together consume approximately 1,200 gates — about 4% of the total DSU. The barrel shifter alone is more than half of this; the FSMs are negligible.

BLOCK IX

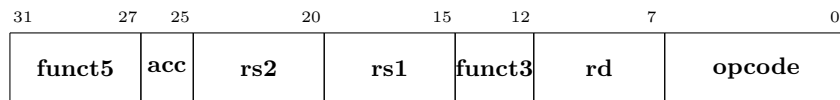
Custom-0 Instruction Set

IX.I Encoding Format

All ten DSU instructions occupy the RISC-V Custom-0 opcode space (0x0B). They use a modified R-type encoding that fits within the standard RISC-V instruction format, so the existing assembler can emit them with minimal extension.

IX.I.1 Bit field layout

The 32-bit instruction word is partitioned as follows:



IX.I.2 Field semantics

funct5 (5 bits) Selects one of 10 DSU instructions. 22 codepoints are reserved and trap as illegal-instruction.

acc (2 bits) Selects target accumulator. 11 is legal only with MACCLEAR.

rs2, rs1 (5 bits each) Standard RISC-V R-type register fields, position-matching the base ISA.

funct3 (3 bits) Fixed 000. The decoder uses this as a sanity check that the instruction is a valid DSU op; values other than 000 trap as illegal-instruction.

rd (5 bits) Destination GPR. Used only by MACREAD_LO, MACREAD_HI, and MACSHIFT. Ignored for all other instructions.

opcode (7 bits) Custom-0 = 0001011.

IX.II funct5 Assignments

IX.III Accumulator Select

IX.IV Instruction Reference

IX.IV.1 MAC_SEL — Multiply-Accumulate

IX.IV.2 MACSUB — Multiply-Subtract

IX.IV.3 MACCLEAR — Clear Accumulator

Table IX.1: funct5 instruction encoding.

funct5	Instruction	Latency (cycles)
00000	MAC_SEL	2
00001	MACSUB	2
00010	MACCLEAR	1
00011	MACLOAD	1
00100	MACREAD_LO	1
00101	MACREAD_HI	1
00110	MACDOT	2
00111	MACSHIFT	1
01000	MACSAT	1
01001	MACABS	2
01010–11111	reserved	illegal-instruction trap

Table IX.2: acc field encoding.

acc[1:0]	Target	Legal with
00	ACC_FX (MAC 0)	all instructions
01	ACC_FY (MAC 1)	all instructions
10	ACC_MAG (MAC 2)	all instructions
11	broadcast (all three)	MACCLEAR only (illegal-instruction trap otherwise)

IX.IV.4 MACLOAD — Load Accumulator**IX.IV.5 MACREAD_LO — Read Accumulator Low Half****IX.IV.6 MACREAD_HI — Read Accumulator High Half****IX.IV.7 MACDOT — Dot-Product Multiply-Accumulate****IX.IV.8 MACSHIFT — Arithmetic Right-Shift Accumulator****IX.IV.9 MACSAT — Saturate Accumulator****IX.IV.10 MACABS — Absolute-Value Multiply-Accumulate****IX.V Illegal-Instruction Conditions**

The decoder raises `illegal_instr` when any of the following holds and `dsu_en` is asserted:

- `funct5` is in the reserved range 01010–11111.
- `acc_sel` is 11 and `funct5` is not MACCLEAR.
- `funct3` is non-zero.

IX.VI Programming Example: APF Inner Loop

The following assembly sequence accumulates the X and Y components of repulsion from one neighbour drone whose relative position (`dx`, `dy`) is in registers `x10`, `x11` and whose scale factor is in `x12`:

Table IX.3: MAC_SEL instruction reference.

Field	Value
Mnemonic	MAC_SEL
funct5	00000
Latency	2 cycles
Syntax	MAC_SEL rs1, rs2, acc
Semantics	$ACC[acc] \leftarrow ACC[acc] + sext48(rs1[15:0] \times rs2[15:0])$
Notes	Primary APF inner-loop instruction. <code>acc_sel=11</code> is illegal.

Table IX.4: MACSUB instruction reference.

Field	Value
Mnemonic	MACSUB
funct5	00001
Latency	2 cycles
Syntax	MACSUB rs1, rs2, acc
Semantics	$ACC[acc] \leftarrow ACC[acc] - sext48(rs1[15:0] \times rs2[15:0])$
Notes	Used for attractive-field computation in APF. <code>acc_sel=11</code> is illegal.

```
# Per-neighbour APF contribution
MAC_SEL x10, x12, acc=0    # ACC_FX += dx * scale
MAC_SEL x11, x12, acc=1    # ACC_FY += dy * scale
```

Two instructions per neighbour, four cycles each (two cycles execute, no stall between because they target different accumulators and have no data dependency). Ten neighbours: 80 cycles. The outer loop maintenance and per-neighbour distance computation bring the total APF cost to roughly 300 cycles per control-loop iteration, well within the 200,000-cycle budget at 200 MHz.

Table IX.5: MACCLEAR instruction reference.

Field	Value
Mnemonic	MACCLEAR
funct5	00010
Latency	1 cycle
Syntax	MACCLEAR <i>acc</i>
Semantics	$ACC[acc] \leftarrow 0$
Notes	<i>acc_sel=11</i> clears all three accumulators in one cycle (broadcast).

Table IX.6: MACLOAD instruction reference.

Field	Value
Mnemonic	MACLOAD
funct5	00011
Latency	1 cycle
Syntax	MACLOAD <i>acc</i> , <i>rs1</i>
Semantics	$ACC[acc] \leftarrow \text{sext48}(rs1[31:0])$
Notes	Initialise accumulator with a non-zero starting value. <i>acc_sel=11</i> is illegal.

Table IX.7: MACREAD_LO instruction reference.

Field	Value
Mnemonic	MACREAD_LO
funct5	00100
Latency	1 cycle
Syntax	MACREAD_LO <i>rd</i> , <i>acc</i>
Semantics	$rd \leftarrow ACC[acc][31:0]$
Notes	Reads low 32 bits of accumulator. Does not modify ACC. <i>acc_sel=11</i> is illegal.

Table IX.8: MACREAD_HI instruction reference.

Field	Value
Mnemonic	MACREAD_HI
funct5	00101
Latency	1 cycle
Syntax	MACREAD_HI <i>rd</i> , <i>acc</i>
Semantics	$rd \leftarrow \text{sext32}(ACC[acc][47:32])$
Notes	Reads upper 16 bits, sign-extended to 32 bits. Does not modify ACC. <i>acc_sel=11</i> is illegal.

Table IX.9: MACDOT instruction reference.

Field	Value
Mnemonic	MACDOT
funct5	00110
Latency	2 cycles
Syntax	MACDOT rs1 , rs2 , acc
Semantics	$ACC[acc] += rs1[31:16] \times rs2[31:16] + rs1[15:0] \times rs2[15:0]$
Notes	Two multiply-accumulates per instruction. Used in EKF matmul. acc_sel=11 is illegal.

Table IX.10: MACSHIFT instruction reference.

Field	Value
Mnemonic	MACSHIFT
funct5	00111
Latency	1 cycle
Syntax	MACSHIFT rd , acc , imm
Semantics	$rd \leftarrow (ACC[acc] \gg_{arith} imm)[31:0]$
Notes	Arithmetic right-shift. Does not modify ACC. imm is 5-bit unsigned. acc_sel=11 is illegal.

Table IX.11: MACSAT instruction reference.

Field	Value
Mnemonic	MACSAT
funct5	01000
Latency	1 cycle
Syntax	MACSAT acc
Semantics	$ACC[acc] \leftarrow \text{clamp32}(ACC[acc]);$ sticky overflow flag set if clamped
Notes	Clamps to 32-bit signed range. acc_sel=11 is illegal.

Table IX.12: MACABS instruction reference.

Field	Value
Mnemonic	MACABS
funct5	01001
Latency	2 cycles
Syntax	MACABS rs1 , rs2 , acc
Semantics	$ACC[acc] += rs1[15:0] \times rs2[15:0] $
Notes	Two's-complement abs on each operand before multiply. acc_sel=11 is illegal.

BLOCK X

Verification, Constraints, and Sign-Off

X.I Timing Constraints

X.I.1 Critical paths

The two main critical paths inside the DSU are:

1. **Cycle 1 (multiplier):** `rs1_data / rs2_data` → abs pre-stage → multiplier → 3:2 compressor → product register. Estimated 1.7 ns.
2. **Cycle 2 (accumulator update):** product register → sign-extend → CSA pre-stage (MACDOT only) → Kogge-Stone 48-bit adder → accumulator-write mux → accumulator register. Estimated 1.5 ns (two-input mode), 1.7 ns (MACDOT).

Both paths fit inside the 5 ns clock period at 200 MHz with ~3 ns of margin for clock skew, setup time, wire delay, and PVT variation.

X.I.2 Fallback frequency

If first-pass synthesis reports negative slack on the critical path, the fallback option is to halve the clock to 100 MHz. The DSU has no internal clock division and would run on the slower core clock automatically.

X.II Power Management

X.II.1 Clock gating policy

Two levels of clock gating are implemented:

- **Fine-grained per-MAC:** Each MAC's product register and accumulator register are clock-gated when this MAC's `en` is low. In typical operation, two of three MACs are gated every cycle.
- **Coarse-grained DSU-wide:** The entire DSU is clock-gated when no Custom-0 instruction has been seen for ≥ 4 cycles. The gating cell auto-wakes on `dsu_en`.

Per-aux-unit clock gating is left to synthesis-tool discretion — the aux path is combinational and benefits more from operand isolation than from explicit clock gating.

X.II.2 Estimated power

Static power is dominated by the three accumulator banks (3×48 retention flops + clock-gate cells). Dynamic power is concentrated in the inner APF loop (~300 cycles every 1 ms) at approximately 30 % MAC duty cycle. Estimated dynamic contribution: ~1.5 mW at 200 MHz, 28 nm typical corner — to be refined after PrimeTime PX analysis.

X.III Verification Hooks

X.III.1 Built-in observability

- **Debug accumulator taps** (`dsu_dbg_acc_0/1/2`) — continuous 48-bit view of every accumulator, observable from the debug module without injecting instructions.
- **Sticky overflow flag** (`dsu_overflow`) — saturation events are recorded permanently until firmware clears the flag.
- **Illegal-instruction trap** (`illegal_instr`) — malformed DSU instructions are caught at decode and signalled to the trap controller.

X.III.2 Required directed tests

Table X.1: Required directed tests.

Test ID	Scenario
T01	MAC_SEL followed by MACSUB targeting same accumulator.
T02	MACDOT halfword packing endianness (high $\times$ high and low $\times$ low ordering).
T03	MACSHIFT sign-bit preservation across all values of <code>imm</code> .
T04	MACSAT positive clamp at <code>0x0000_0000_FFFF_FFFF</code> .
T05	MACSAT negative clamp at <code>0xFFFF_FFFF_0000_0000</code> .
T06	MACABS double-negative case (both operands < 0).
T07	MACCLEAR broadcast (<code>acc_sel=11</code>).
T08	Back-to-back 2-cycle MACs (stall behaviour).
T09	EX→EX forwarding into MAC_SEL rs1.
T10	EX→EX forwarding into MAC_SEL rs2.
T11	Trap during cycle 2 of a 2-cycle MAC (accumulator unchanged).
T12	All 22 reserved <code>funct5</code> values trap as illegal-instruction.
T13	<code>acc_sel=11</code> with each non-MACCLEAR <code>funct5</code> traps.
T14	Non-zero <code>funct3</code> traps as illegal-instruction.
T15	Saturation overflow flag stickiness across non-MACSAT instructions.
T16	CSR-clear of overflow flag during simultaneous MACSAT clamp.
T17	APF golden-model regression on 100 random neighbour configurations.
T18	EKF golden-model regression on full matrix-multiply trace.

X.III.3 Reference model

The APF golden Python model produces expected accumulator state at every instruction boundary. Direct RTL comparison against this model forms the per-block verification methodology for the DSU.

X.IV Gate Count Summary

The DSU consumes approximately 30,000 gates pre-synthesis. The six 16×16 multipliers dominate at $\sim 55\%$ of total area; the three accumulator adders and accumulator registers together account for $\sim 20\%$; the aux blocks and control logic together are $\sim 5\%$.

Table X.2: Total DSU gate-count rollup (pre-synthesis estimates).

Component	Gates
Six 16×16 multipliers (3 MACs × 2 each)	~16,800
Three 48-bit Kogge-Stone adders (with CSA pre-stage)	~4,590
Three accumulator registers (48 flops each + clock-gate)	~3,000
Three product registers (33+33 flops each, clock-gated)	~4,200
Three abs pre-stages (per-MAC, both operands)	~300
Three operand routers + operand isolation	~270
Three 3:2 compressors (for MACDOT)	~450
Three accumulator-write muxes	~600
Cluster-level enable decoder + sat router + output mux	~250
MAC cluster subtotal	~30,460
DSU instruction decoder	~200
Barrel shifter	~720
Saturation unit	~200
Read-back mux	~100
Result selector mux	~150
Stall FSM + overflow flag + rd_valid decode	~30
DSU total	~31,860

X.V Sign-Off Checklist

Before this design specification is considered closed for RTL implementation, the following items must be confirmed:

- ☐ All ten Custom-0 instructions implemented and matched against the APF golden Python model.
- ☐ Illegal-instruction trap fires for all 22 reserved `funct5` codepoints.
- ☐ Illegal-instruction trap fires for `acc_sel=11` with any `funct5` other than `MACCLEAR`.
- ☐ `dsu_busy` asserts exactly once per 2-cycle instruction and never for 1-cycle instructions.
- ☐ Flush during cycle 2 of a 2-cycle instruction leaves accumulator state unchanged.
- ☐ `MACDOT` halfword packing endianness matches the toolchain's emission.
- ☐ `MACSHIFT` preserves sign of all-bits-set accumulator values.
- ☐ `MACSAT` clamps at both positive and negative 32-bit signed boundaries.
- ☐ `MACABS` produces correct result for the double-negative case (both operands negative).
- ☐ `MACCLEAR` with `acc_sel=11` clears all three accumulators in one cycle.
- ☐ `EX→EX` forwarding into a DSU instruction's `rs1/rs2` produces the same result as a non-forwarded equivalent.

- ☐ Trap during cycle 2 of a 2-cycle instruction restores PC correctly and accumulator state matches pre-trap value.
- ☐ Synthesis closure at 200 MHz on the 28 nm PDK with positive slack on all critical paths.

Bibliography